

Abilitare il servizio Pub-Sub

- Il primo passo è quello di installare alcune estensioni al modulo node-opcua, relative al protocollo PubSub:
 - \$ npm install node-opcua-pubsub-server
 - \$ npm install node-opcua-pubsub-expander
 - \$ npm install node-opcua-types
 - \$ npm install node-opcua-file-transfer
- Nel programma si dovranno caricare tutti i moduli (librerie) che servono nel programma, utilizzando "require":

```
const opcua = require("node-opcua");  
const pubsubexpander = require("node-opcua-pubsub-expander");  
const pubsubserver = require("node-opcua-pubsub-server");  
const opcuatype = require("node-opcua-types");
```

Abilitare il servizio Pub-Sub

- Ci sono due operazioni essenziali per trasformare il nostro server OPCUA in un publisher PubSub-ready:
 - Caricare tutti i **parametri di configurazione** del publisher visti a lezione
 - Eseguire il metodo **installPubSub** in modo sincrono
- Tutti ciò dopo **await server.initialize();** del codice del nostro server.

```
const server = new opcua.OPCUAServer ({
    port: 4334,
    userManager,
    allowAnonymous: false,
    resourcePath: "/UA/MyLittleServer",
});
await server.initialize();
//...altro codice ...

//"enable pub-sub service"
const configuration = getPubSubConfiguration();

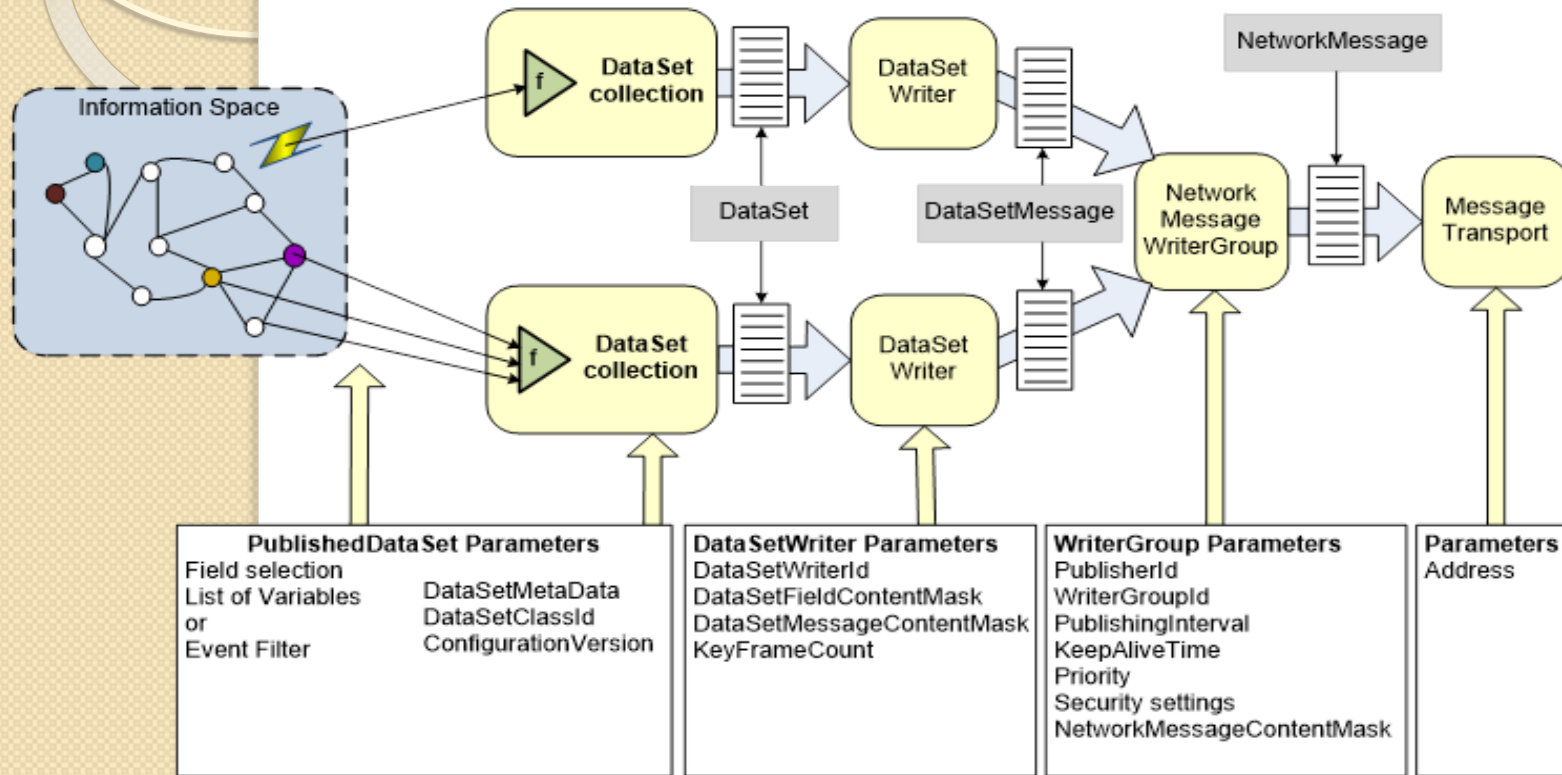
await installPubSub(server, {
    configuration,
});
```

**Configurazione dei
parametri del
Publisher**

Creazione dell'oggetto
PublishSubscribe e del binding
di tutti i metodi e servizi

OPC UA PubSub Model

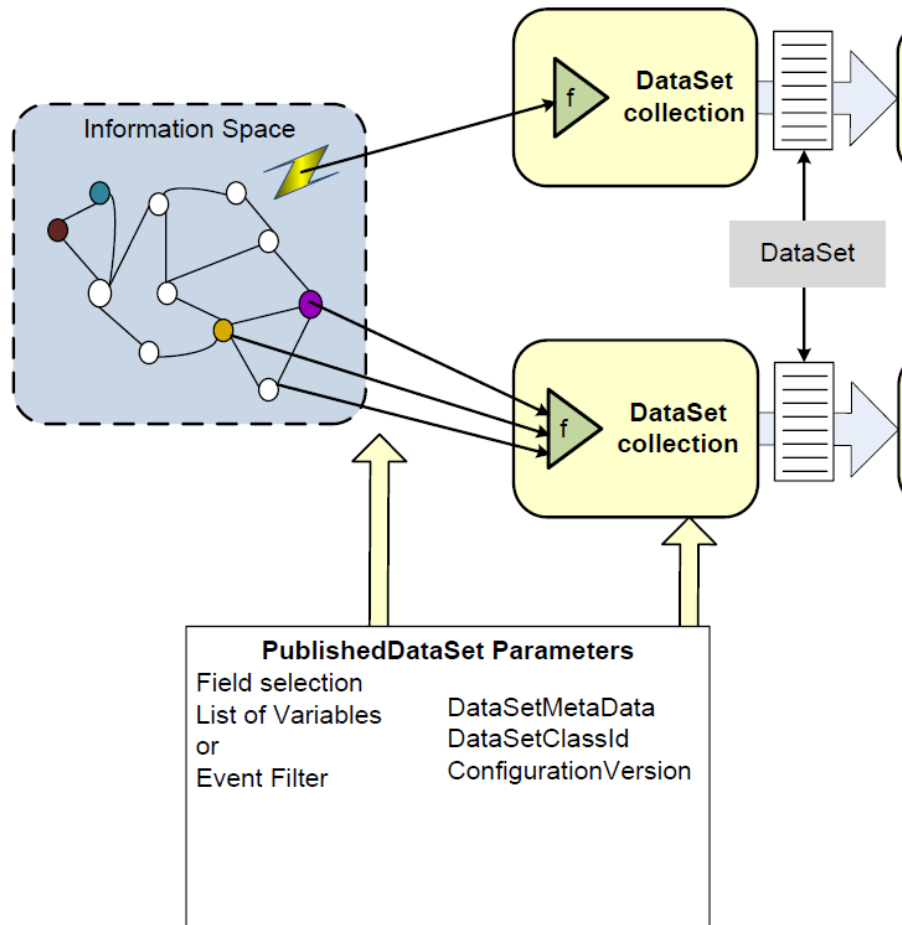
Publisher



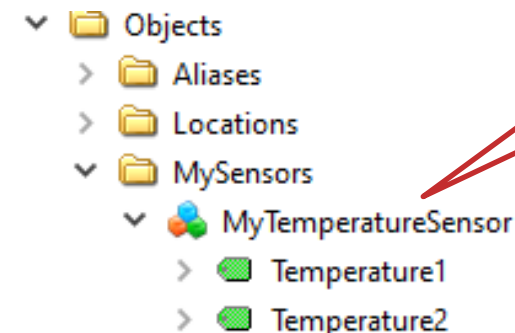
- **opcua.type.PubSubConfigurationDataType** è un **tipo di dato strutturato** definito nello standard **OPC UA**, usato per rappresentare **l'intera configurazione di un sistema PubSub**
- Questo tipo di dato è una struttura complessa che include:
 - **PublishedDataSets** → quali dati vengono messi a disposizione
 - **Connections** → come i nodi sono collegati (rete, protocollo, ecc.)
 - **WriterGroups** → gruppi che pubblicano dati

OPC UA PubSub Model

Publisher



- **Primo passo.** Collezione dei dati da pubblicare.
- La collezione produce valori per il DataSet da parte del DataSet Collector
- La configurazione è definita dal **PublishedDataSet**, che contiene, tra gli altri:
 - **DataSetMetaData**
 - **Lista delle Variabili:** NodeId, SamplingInterval, Deadband
 - **Lista Eventi:** NodeId Event Notifier, ContentFilter
- Supporremo che la lista delle variabili sia unicamente formata da due DataVariable Value



temperatureSensor

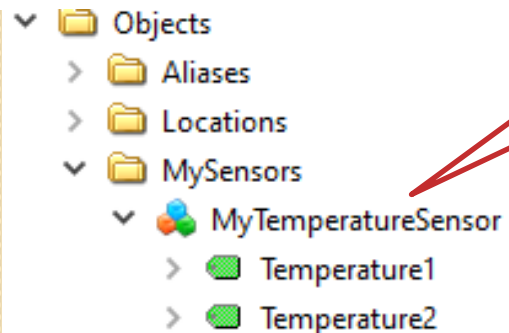
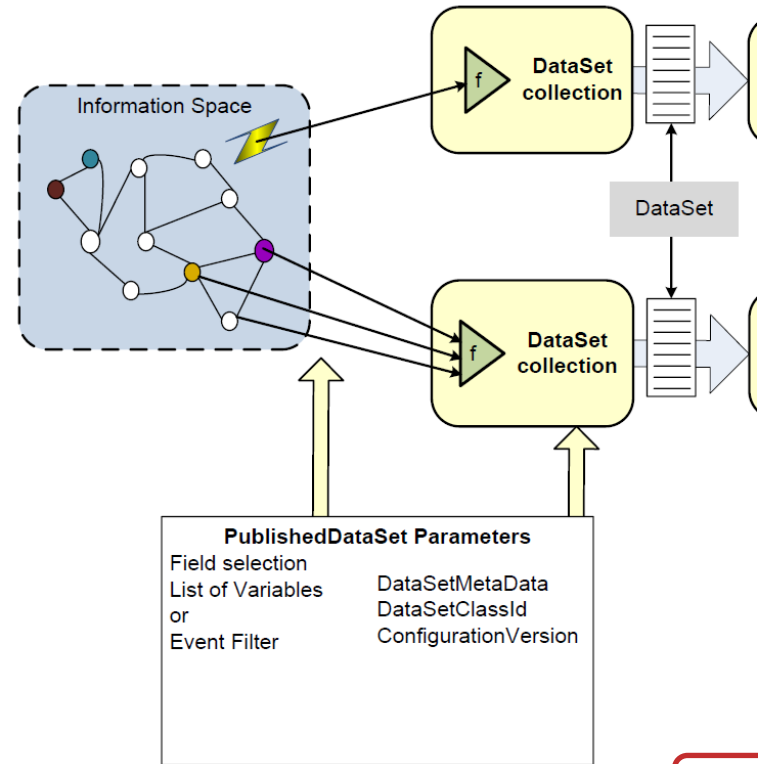
OPC UA PubSub Model Publisher

PublishedDataSet

- **Name**="PublishedDataSet1"
- DataSetMetaData
 - Elenco dei Fields
- DataSetSource
 - Lista di Variabili (attributo, nodeId, SamplingInterval)

```
function createPublishedDataSet(temperatureSensor) {  
  const publishedDataSet = {  
    name: "PublishedDataSet1",  
    dataSetMetaData: {  
      fields: [  
        {  
          name: "Sensor.Temperature1",  
          builtInType: opcua.DataType.Double,  
          dataType: opcua.resolveNodeId("Double"),  
        },  
        {  
          name: "Sensor.Temperature2",  
          builtInType: opcua.DataType.Double,  
          dataType: opcua.resolveNodeId("Double"),  
        },  
      ],  
    },  
  },  
};
```

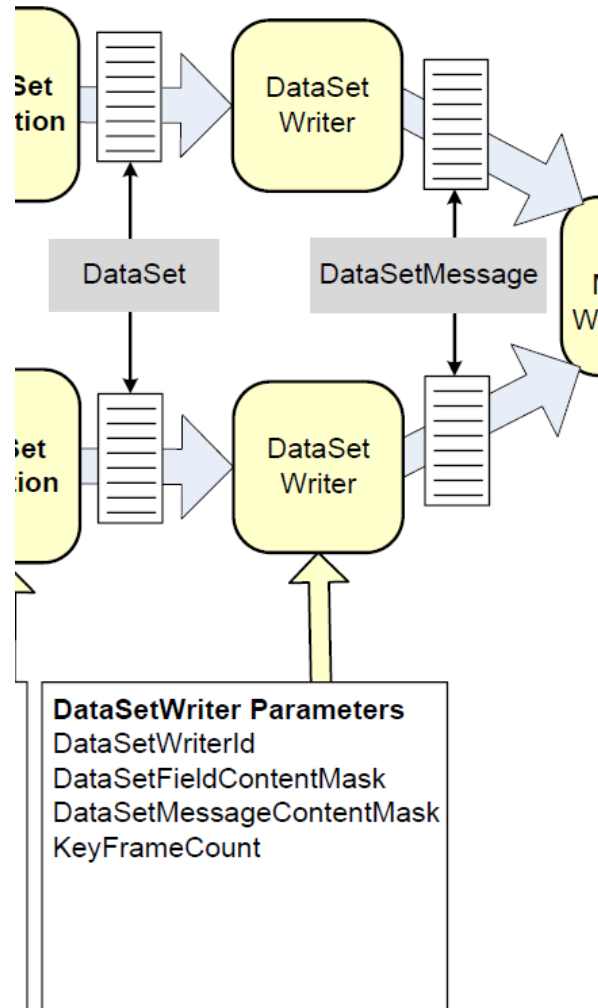
```
    dataSetSource: new opcuaType.PublishedDataItemsDataType({  
      publishedData: [  
        {  
          attributeId: opcua.AttributeIds.Value,  
          samplingIntervalHint: 100,  
          publishedVariable: temperatureSensor.temperature1.nodeId,  
        },  
        {  
          attributeId: opcua.AttributeIds.Value,  
          samplingIntervalHint: 100,  
          publishedVariable: temperatureSensor.temperature2.nodeId,  
        },  
      ],  
    }),  
  };  
  return publishedDataSet;  
}
```



temperatureSensor

OPC UA PubSub Model

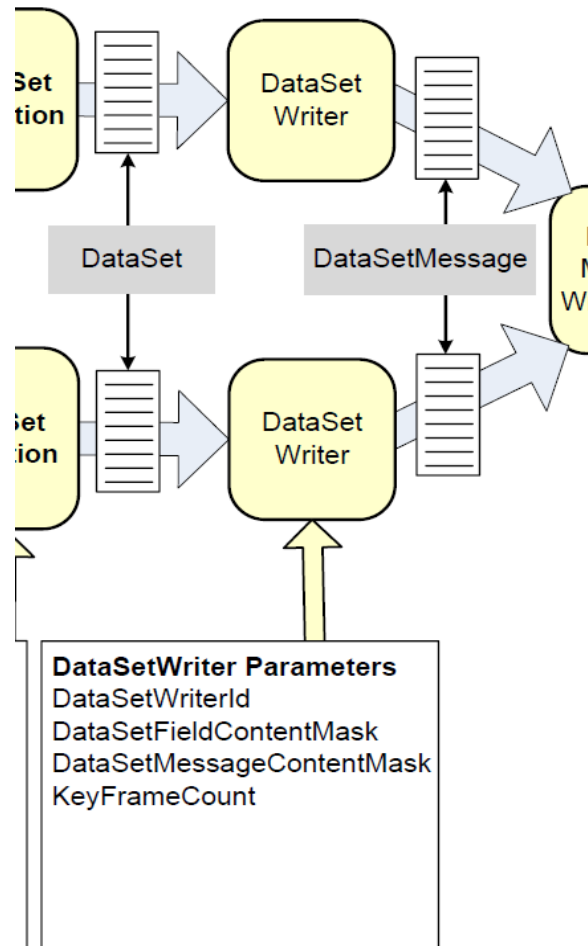
Publisher



- **Il secondo passo** è realizzato dal DataSetWriter che crea il DataSetMessage a partire dai DataSet
- La creazione di un DataSetMessage è guidata dal **DataSetWriter**, che contiene tra gli altri:
 - **DataSetWriterId**
 - **DataSetFieldContentMask**
 - Variant, DataValue, RawData

OPC UA PubSub Model

Publisher



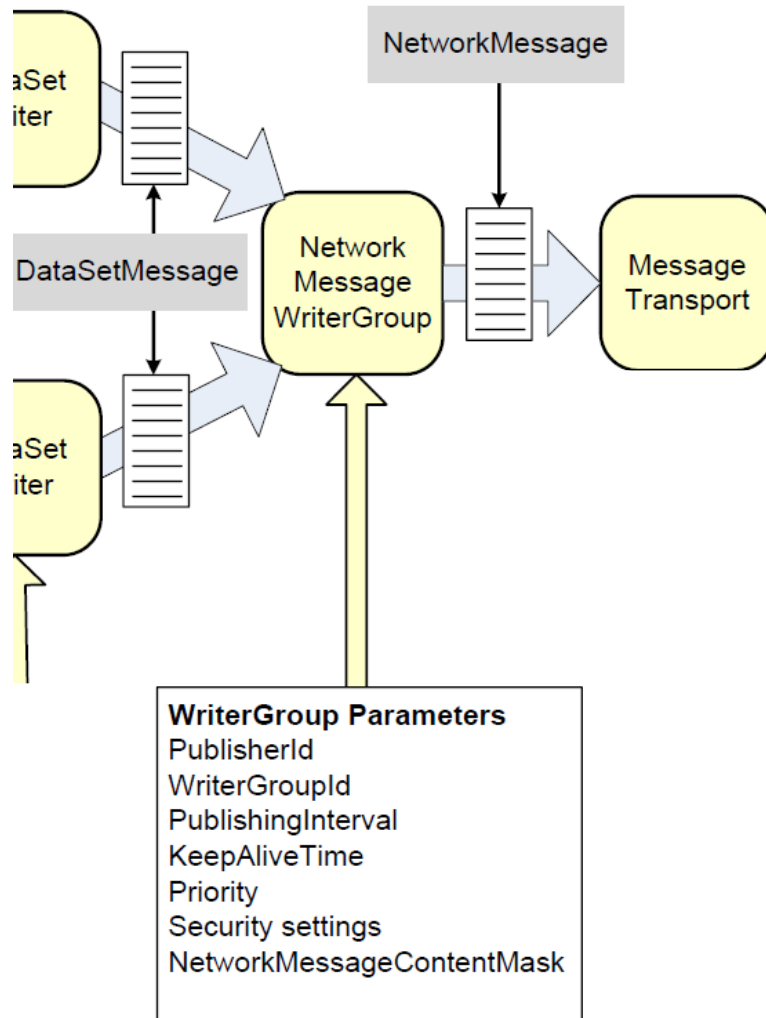
```
// "create the dataset writer"
const dataSetWriter = {
  dataSetFieldContentMask: opcua.type.DataSetFieldContentMask.None,
  dataSetName: "PublishedDataSet1",
  dataSetWriterId: 1,
  enabled: true,
  name: "dataSetWriter1",
  messageSettings: {
    dataSetMessageContentMask:
      opcua.type.JsonDataSetMessageContentMask.DataSetWriterId |
      opcua.type.JsonDataSetMessageContentMask.MetadataVersion,
  },
  transportSettings: {
    queueName: "temperature-sensors",
  },
};
```

DataSetWriter

- **DataSetFieldContentMask.None=Variant**
- DataSetName="PublishedFiDataSetI" nome del DataSet pubblicato
- DataSetWriterId=1
- Name ="dataSetWriter1"
- DataSetMessageContentMask=include WriterId e MetadataVersion
- Queue name (topic)="temperature-sensors"

OPC UA PubSub Model

Publisher

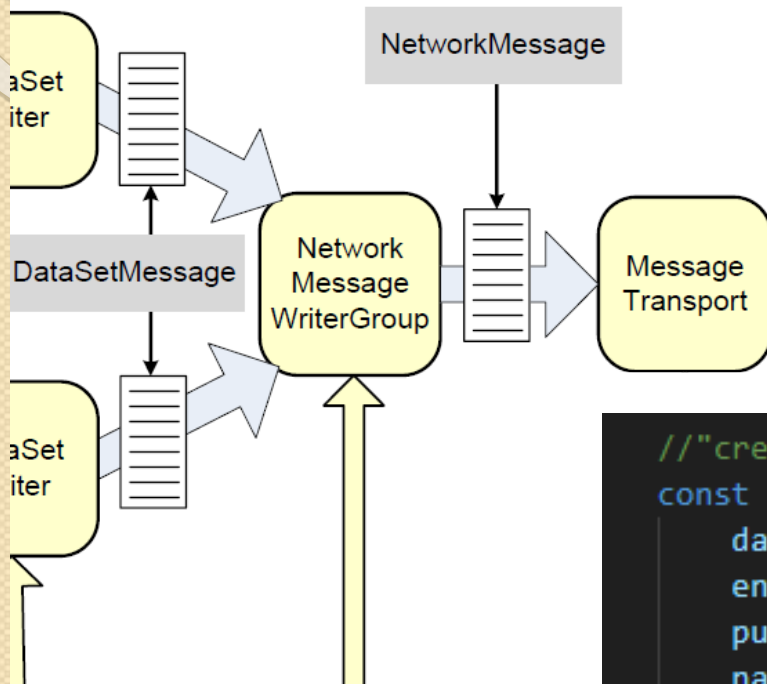


- **Un altro passo** è rappresentato dalla creazione di un **NetworkMessage**
- Il processo di creazione è governato da diversi parametri del **WriterGroup**
 - **PublishingInterval**: frequenza di creazione ed invio dei **NetworkMessage**

OPC UA PubSub Model

Publisher

Pattern di comunicazione possibili:
AtMostOnce= nessuna garanzia di ricevere correttamente il messaggio trasmesso
AtLeastOnce= il messaggio può essere ritrasmesso se non correttamente ricevuto (possono arrivare duplicati, ma almeno uno arriva)



WriterGroup Parameters
PublisherId
WriterGroupId
PublishingInterval
KeepAliveTime
Priority
Security settings
NetworkMessageContentMask

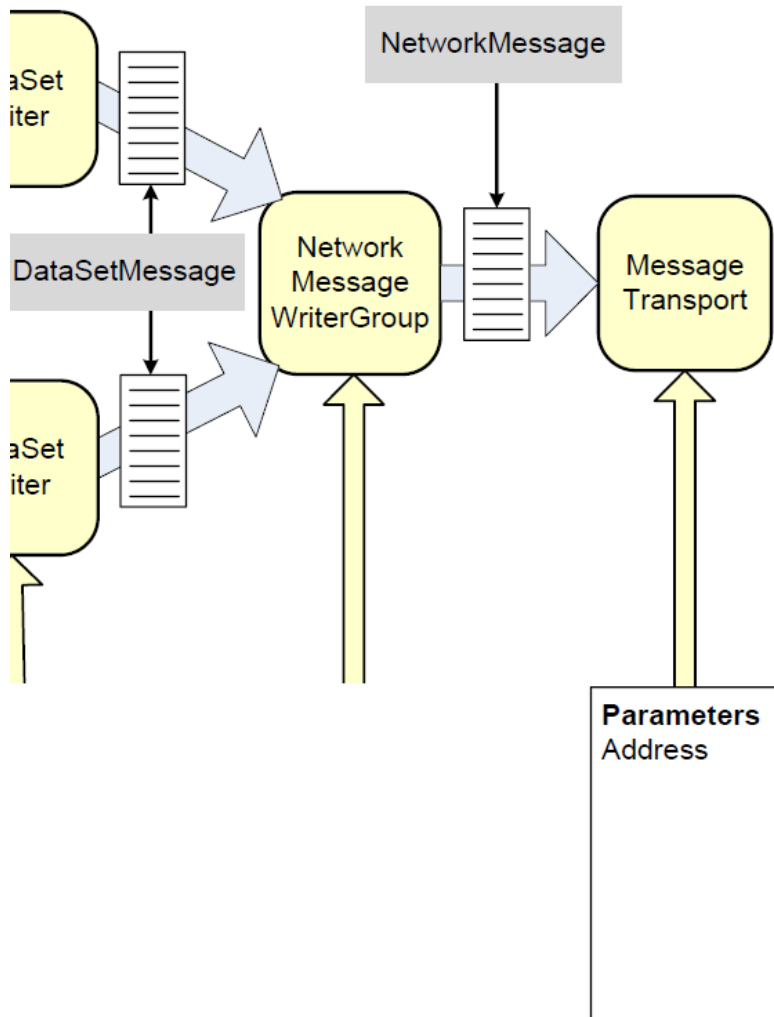
WriterGroup

- Name = "WriterGroup1"
- PublishingInterval = 1000
- **MessageSettings** = contiene il **PublisherId**
- Transport Settings = **AtMostOnce** (equivalente a QoS 0 in MQTT)

```
// "create the writer group";  
const writerGroup = {  
  dataSetWriters: [dataSetWriter],  
  enabled: true,  
  publishingInterval: 1000,  
  name: "WriterGroup1",  
  messageSettings: {  
    networkMessageContentMask: opcuaType.JsonNetworkMessageContentMask.PublisherId,  
  },  
  transportSettings: {  
    requestedDeliveryGuarantee: opcuaType.BrokerTransportQualityOfService.AtMostOnce,  
  },  
};
```

OPC UA PubSub Model

Publisher



```
const connection = new pubsubexpander.MyMqttJsonPubSubConnectionDataType({
  enabled: true,
  name: "Connection1",
  transportProfileUri: pubsubexpander.Transport.MQTT_JSON,
  address: {
    url: mqttEndpoint,
  },
  writerGroups: [writerGroup],
  readerGroups: []
});
```

- **Ultimo passo** è rappresentato dalla creazione di un oggetto di tipo **MyMqttJsonPubSubConnectionDataType**, che definisce il **protocollo di trasporto** (MQTT, UDP...), specifica l'**indirizzo di destinazione del middleware**, contiene i **WriterGroup**
- **TransportProfileUri**. Il parametro TransportProfileUri indica il mapping con il protocollo di trasporto e il message mapping utilizzato.
 - Nel nostro caso: MQTT/JSON
- **Address**. Il parametro Address contiene l'indirizzo da usare per il communication middleware.
 - Nel nostro caso il Broker:

```
const mqttEndpoint = "mqtt:broker.emqx.io";
```

OPC UA PubSub Model

Publisher

- Nome del codice in Node.js **"myserver_pubsub.js"**
- Eseguirlo
- Il programma utilizza il broker **"broker.emqx.io"**
- Per poterlo testare, è possibile utilizzare il client
"https://mqttx.app/web-client#/"
- Creare una nuova connessione (Cliccando su “New Connection” e inserire nel campo Name una qualunque stringa), lasciando tutti gli altri parametri così come sono
- Iscrivere a un topic (Subscribe). Ricordarsi che nel programma abbiamo scelto come topic il nome **"temperature-sensors"**

OPC UA PubSub Model

Publisher

- Sarà possibile vedere tutti gli aggiornamenti delle due variabili `Sensor.Temperature1` e `Sensor.Temperature2`

+ New Subscription

temperature-sens... QoS 0

Plaintext

All

2026-05-01 20:52:59:465

Topic: temperature-sensors QoS: 0

```
{ "MessageId": "9DBD1532-240C-526C-2F0F-0000000000cb", "MessageType": "ua-data", "Messages": [ { "DataSetWriterId": "1", "Payload": { "Sensor.Temperature1": { "Type": 11, "Body": 5.14793039890881 }, "Sensor.Temperature2": { "Type": 11, "Body": 14.136602270660356 } } ] }
```

2026-05-01 20:53:00:482

Topic: temperature-sensors QoS: 0

```
{ "MessageId": "9DBD1532-240C-526C-2F0F-0000000000cc", "MessageType": "ua-data", "Messages": [ { "DataSetWriterId": "1", "Payload": { "Sensor.Temperature1": { "Type": 11, "Body": 5.327307080159298 }, "Sensor.Temperature2": { "Type": 11, "Body": 14.09721704413292 } } ] }
```

2026-05-01 20:53:01:490

Topic: temperature-sensors QoS: 0

```
{ "MessageId": "9DBD1532-240C-526C-2F0F-0000000000cd", "MessageType": "ua-data", "Messages": [ { "DataSetWriterId": "1", "Payload": { "Sensor.Temperature1": { "Type": 11, "Body": 5.1122896923038175 }, "Sensor.Temperature2": { "Type": 11, "Body": 14.119924371547837 } } ] }
```

2026-05-01 20:53:02:489

Topic: temperature-sensors QoS: 0

```
{ "MessageId": "9DBD1532-240C-526C-2F0F-0000000000ce", "MessageType": "ua-data", "Messages": [ { "DataSetWriterId": "1", "Payload": { "Sensor.Temperature1": { "Type": 11, "Body": 5.398322466203243 }, "Sensor.Temperature2": { "Type": 11, "Body": 14.287822248356987 } } ] }
```